INTEGRATED TECHNOLOGIES (AML) LAB

(Course Code: 23UPCSC1L08)

A programming laboratory record submitted to Periyar University, Salem

In partial fulfilment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS

By

YOGANATHAN C

[Reg. No: U24PG507CAP030]

DEPARTMENT OF COMPUTER SCIENCE

PERIYAR UNIVERSITY

State University - NAAC 'A++' Grade - NIRF Rank 94

State Public University Rank 40 - SDG Institutions Rank Band: 11-50

SALEM – 636 011

(OCTOBER - 2025)

CERTIFICATE

This is to certify that the Programming Laboratory entitled "**INTEGRATED TECHNOLOGIES (AML) LAB (23UPCSC1L08)**" is a Bonafide record work done by Mr. /Ms._____ Register No: _____ as partial fulfilment of the requirements for the degree of Master of Computer Applications, in the Department of Computer Science, Periyar University, Salem, during the Academic Year 2025-2026.

Staff In-charge Head of the Department

Submitted for the practical examination held on……..……

Internal Examiner External Examiner

CONTENTS

S.NO	DATE	TITLE	PG NO	SIGNATURE
1		Central Tendency Measures		
2		Linear Regression and Multiple Regression		
3		Logistic Regression using sklearn		
4		Binary Classification Model		
5		Nearest Neighbors and Naive Bayes Algorithm		
6		Decision Tree for Classification		
7		K-Means Algorithm		
8		Image Classifier using CNN in TensorFlow/Keras		
9		Autoencoder in TensorFlow/Keras		
10		Simple LSTM using TensorFlow/Keras		

SOURCE CODE:

```
import pandas as pd
import random
dataset = {
"Marital_status": random.choices(["Yes", "No"], k=600),
"Is_graduate": random.choices(["Yes", "No"], k=600),
"Income": [random.randint(20000, 100000) for _ in range(600)],
"Loan_amount": [random.randint(1000, 50000) for _ in range(600)],
"Term_months": random.choices([12, 24, 36, 48, 60], k=600),
"Age": [random.randint(20, 60) for _ in range(600)],
"Sex": random.choices(["F", "M"], k=600),
"approval_status": random.choices(["Yes", "No"], k=600)
}
data = pd.DataFrame(dataset)
data.to_csv("dataset.csv", index=False)
print("mean :", round(data['Income'].mean(), 3))
print("median :",data['Income'].median())
print("SD :",data['Income'].std())
print("Var :",data['Income'].var())
print("Mode :", data['Income'].mode()[0])
```

OUTPUT:

```
mean : 61182.315
median : 62234.5
SD : 23271.737856547836
Var : 541573782.8638817
Mode : 46516
```

RESULT:

SOURCE CODE:

#Linear Regression

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import r2_score , mean_squared_error

import matplotlib.pyplot as plt

dataset=pd.read_csv("https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv")

dataset.info()

dataset

X=dataset.drop('medv',axis=1)

y=dataset['medv']

x=X[['age']]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=10)

lr=LinearRegression()

lr.fit(x_train,y_train)

result=lr.predict(x_test)

mse = mean_squared_error(y_test, result)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, result)

print(f"Mean Squared Error: {mse:.2f}")

print(f"Root Mean Squared Error: {rmse:.2f}")

print(f"R-squared: {r2:.2f}")

plt.scatter(y_test, result)

plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--', color='red')

plt.xlabel('Actual Values')

plt.ylabel('Predicted Values')
```

```
plt.title('Regression Line')
plt.show()
```

 # Multi Linear Regression

```
x_train,x_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=10)
lr=LinearRegression()
lr.fit(x_train,y_train)
result1=lr.predict(x_test)
mse = mean_squared_error(y_test, result1)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, result1)
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-squared: {r2:.2f}")
plt.scatter(y_test,result1)
plt.plot([min(y_test),max(y_test)],[min(y_test),max(y_test)],linestyle='--',color='red')
plt.show()
```

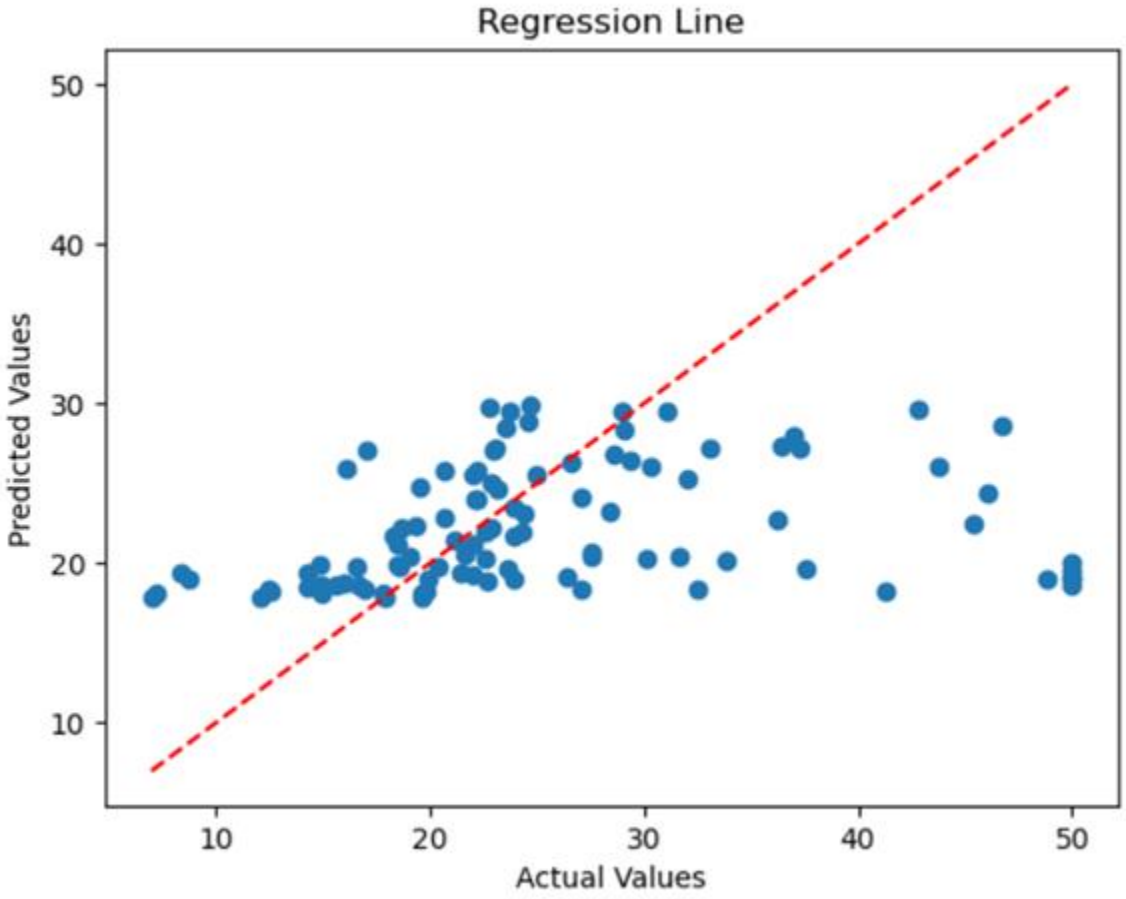
OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64
 9   tax      506 non-null    int64
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

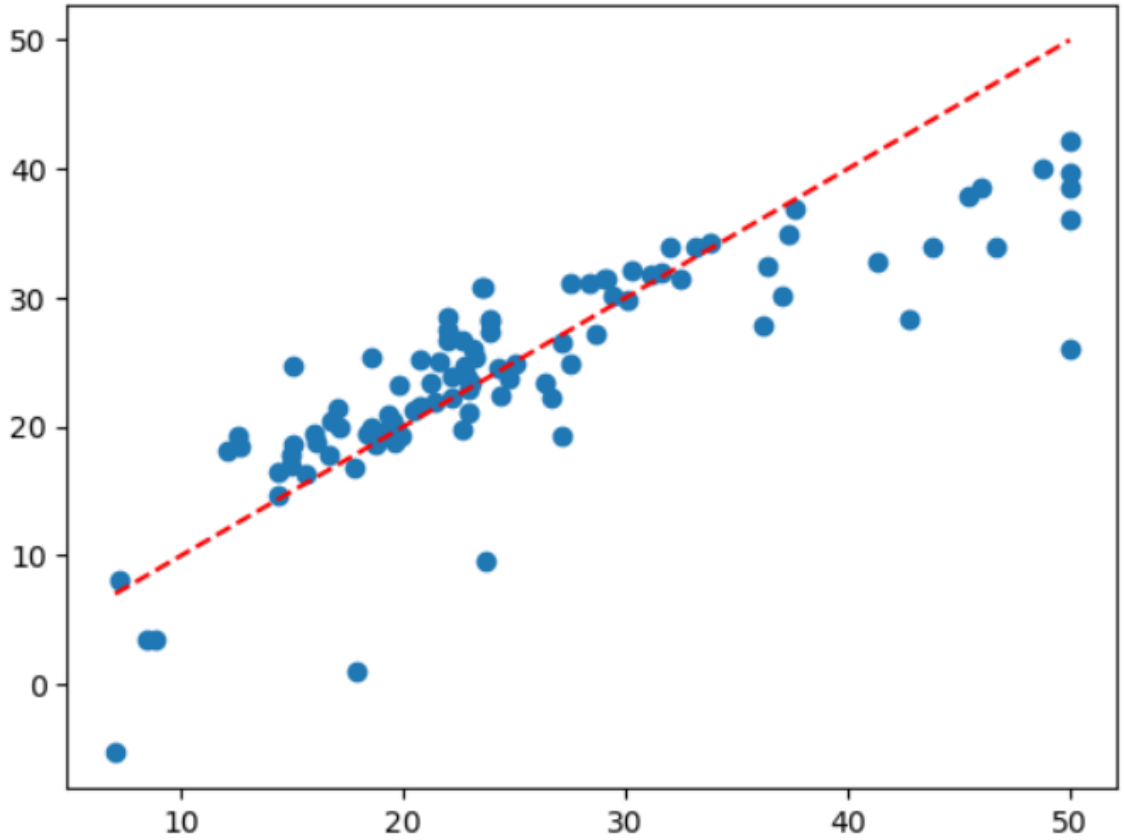
	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

```
Mean Squared Error: 107.93
Root Mean Squared Error: 10.39
R-squared: -0.03
```



```
Mean Squared Error: 34.41
Root Mean Squared Error: 5.87
R-squared: 0.67
```



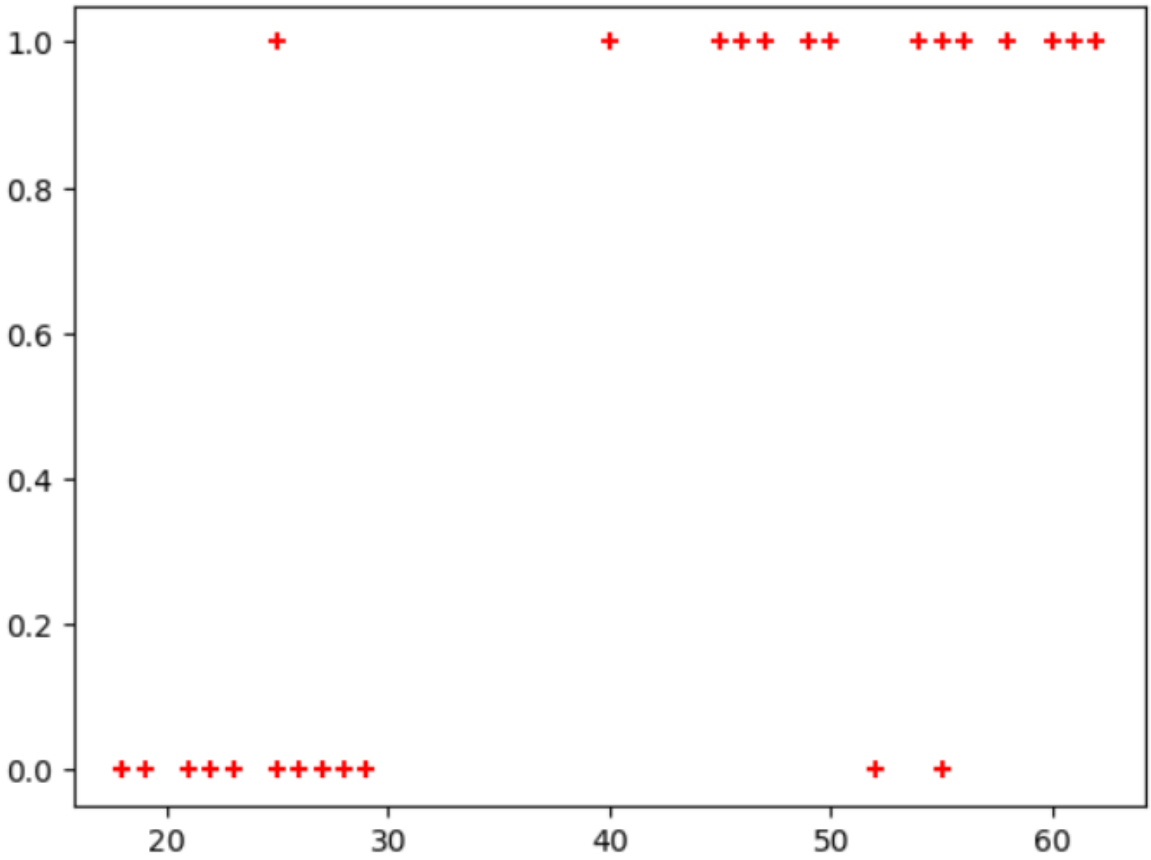
RESULT:

SOURCE CODE:

```
import pandas as pd

from matplotlib import pyplot as plt

df=pd.read_csv("C:/Users/CC-Lab79/Downloads/insurance_data.csv")

df.head()

plt.scatter(df.age,df.bought_insurance,marker='+',color='red')

plt.show()

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test =
train_test_split(df[['age']],df.bought_insurance,train_size=0.8)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)

y_predicted=model.predict(X_test)

model.predict_proba(X_test)

model.score(X_test,y_test)

from sklearn.metrics import classification_report

classification_rep=classification_report(y_test, y_predicted)

print(classification_rep)
```

OUTPUT:

	age	bought_insurance
0	22	0
1	25	0
2	47	1
3	52	0
4	46	1




```
array([[0.01119575, 0.98880425],
       [0.58148802, 0.41851198],
       [0.03631615, 0.96368385],
       [0.98085288, 0.01914712],
       [0.0179866 , 0.9820134 ],
       [0.99259553, 0.00740447]])
```

```
0.5

              precision    recall  f1-score   support

           0       0.33      0.50      0.40         2
           1       0.67      0.50      0.57         4

    accuracy                           0.50         6
   macro avg       0.50      0.50      0.49         6
weighted avg       0.56      0.50      0.51         6
```

RESULT:

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
iris = load_iris()
X = iris.data
y = iris.target
y = np.where(y == 0, 0, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
metrics = {"Accuracy": [], "Precision": [], "Recall": [], "F1 Score": []}
for k in kernels:
    svm = SVC(kernel=k, random_state=42)
    svm.fit(X_train, y_train)
    y_pred = svm.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    metrics["Accuracy"].append(acc)
    metrics["Precision"].append(prec)
    metrics["Recall"].append(rec)
    metrics["F1 Score"].append(f1)
    print(f"\nKernel: {k}")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
```

```
    print(f"Recall   : {rec:.4f}")
    print(f"F1 Score : {f1:.4f}")
x = np.arange(len(kernels))
width = 0.2
plt.bar(x - 0.3, metrics["Accuracy"], width, label='Accuracy')
plt.bar(x - 0.1, metrics["Precision"], width, label='Precision')
plt.bar(x + 0.1, metrics["Recall"], width, label='Recall')
plt.bar(x + 0.3, metrics["F1 Score"], width, label='F1 Score')
plt.xticks(x, kernels)
plt.ylabel("Score")
plt.title("SVM Performance with Different Kernels")
plt.legend()
plt.show()
```

OUTPUT:

Kernel: linear

Accuracy : 1.0000

Precision: 1.0000

Recall : 1.0000

F1 Score : 1.0000

Kernel: poly

Accuracy : 1.0000

Precision: 1.0000

Recall : 1.0000

F1 Score : 1.0000

Kernel: rbf

Accuracy : 1.0000

Precision: 1.0000

Recall : 1.0000

F1 Score : 1.0000

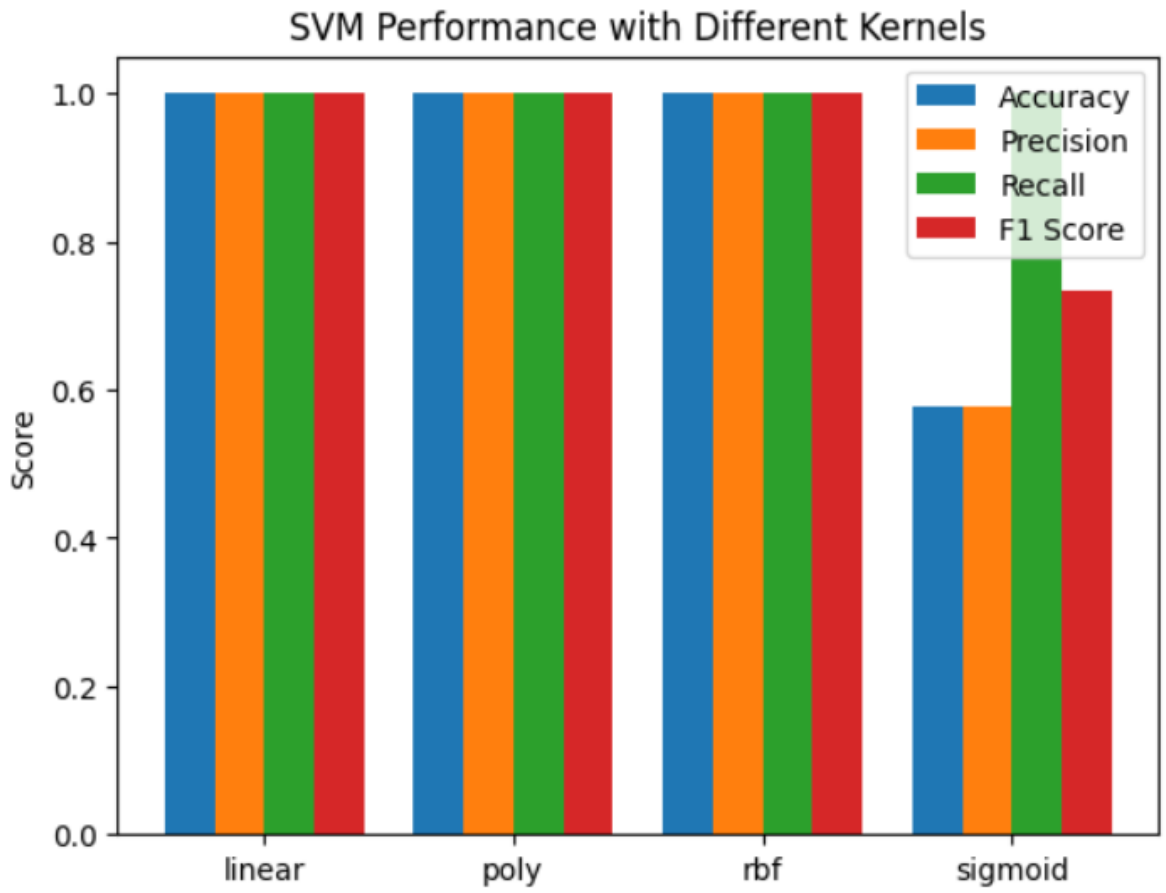
Kernel: sigmoid

Accuracy : 0.5778

Precision: 0.5778

Recall : 1.0000

F1 Score : 0.7324


SVM Performance with Different Kernels
Accuracy
Precision
Recall
F1 Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
linear
poly
rbf
sigmoid

RESULT:

SOURCE CODE:

 # KNN

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn import metrics
from sklearn.neighbors import KNeighborsClassifier
iris=datasets.load_iris()
df=pd.DataFrame(data=np.c_[iris['data'],iris['target']],
            columns=iris['feature_names']+['target'])
df.head()
df["target"].value_counts()
sns.FacetGrid(df,hue="target",height=8).map(plt.scatter,"petal length (cm)","petal width
(cm)").add_legend()
plt.show()
model=KNeighborsClassifier(n_neighbors=3)
model.fit(iris.data, iris.target)
predicted = model.predict(iris.data)
expected= iris.target
model.score(iris.data,iris.target)
print(metrics.confusion_matrix(expected, predicted))
```

 # Naive Bayes

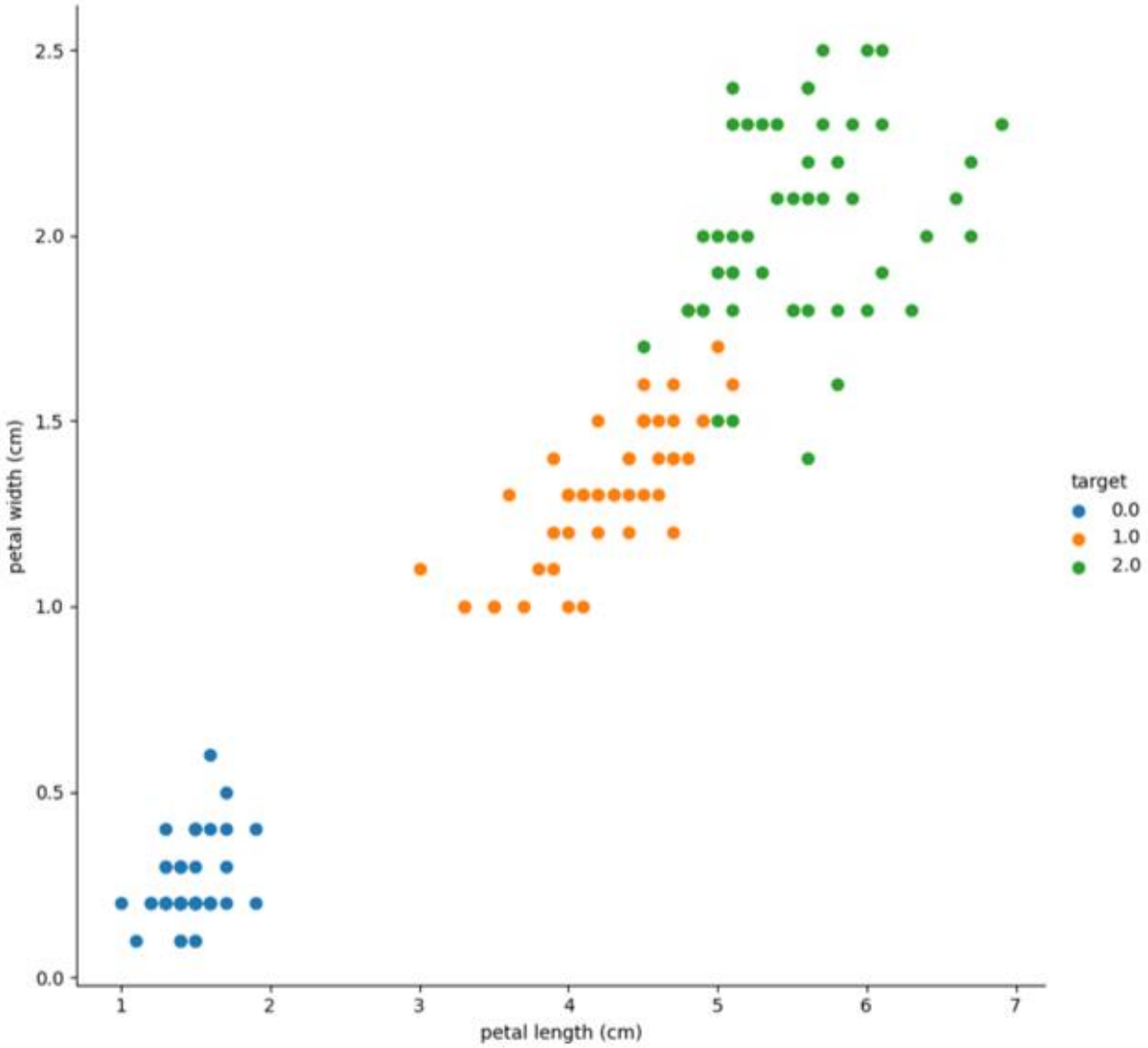
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn import metrics
```

```
from sklearn.naive_bayes import GaussianNB
iris=datasets.load_iris()
df=pd.DataFrame(data=np.c_[iris['data'],iris['target']],
            columns=iris['feature_names']+['target'])
df.head()
sns.FacetGrid(df,hue="target",height=8).map(plt.scatter,"petal length (cm)","petal width
(cm)").add_legend()
plt.show()
model=GaussianNB()
model.fit(iris.data, iris.target)
predicted = model.predict(iris.data)
expected= iris.target
model.score(iris.data,iris.target)
print(metrics.confusion_matrix(expected,predicted))
```

OUTPUT:

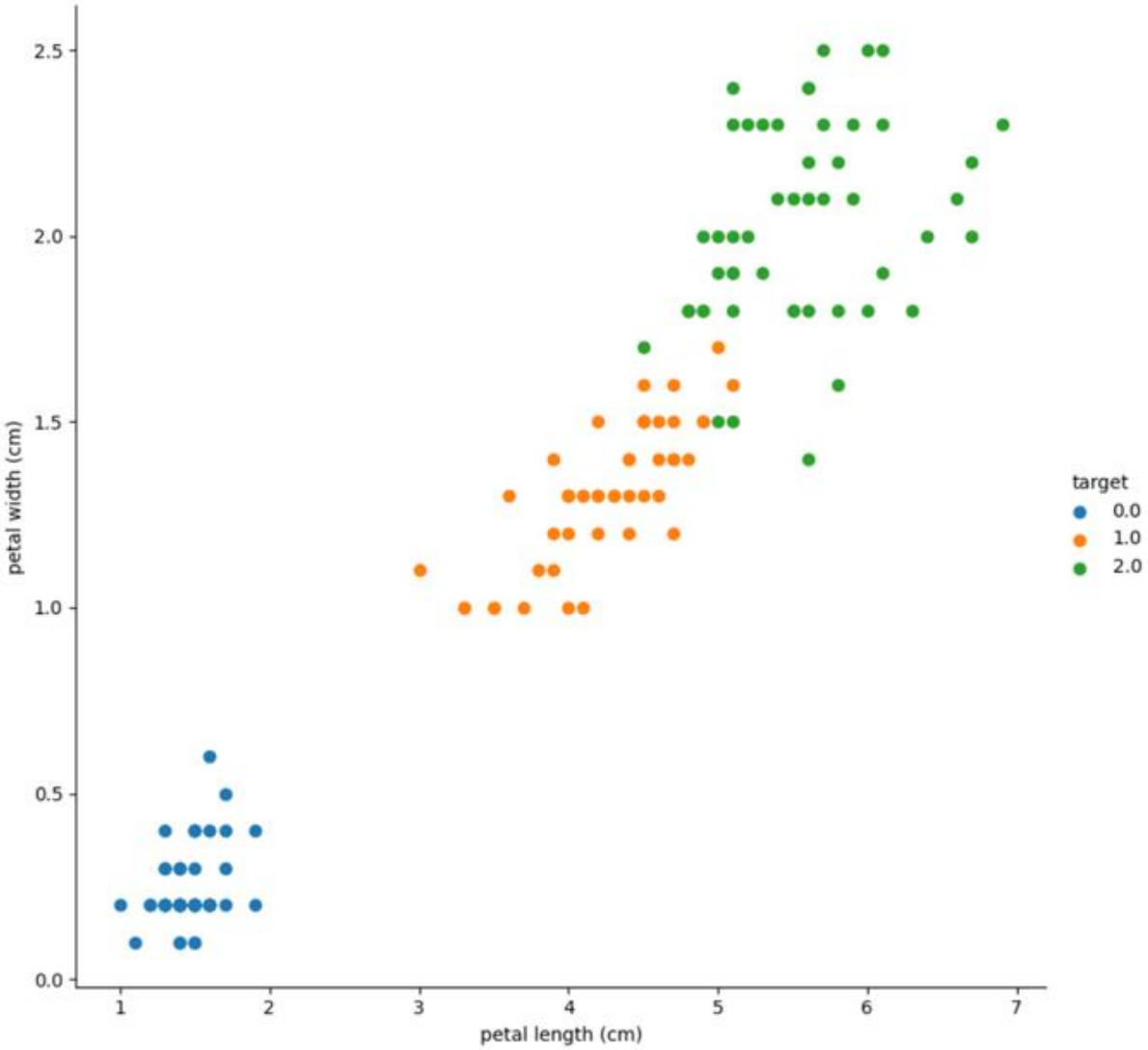
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0

```
target
0.0    50
1.0    50
2.0    50
Name: count, dtype: int64
```



```
0.96

[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0



```
0.96

[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

RESULT:

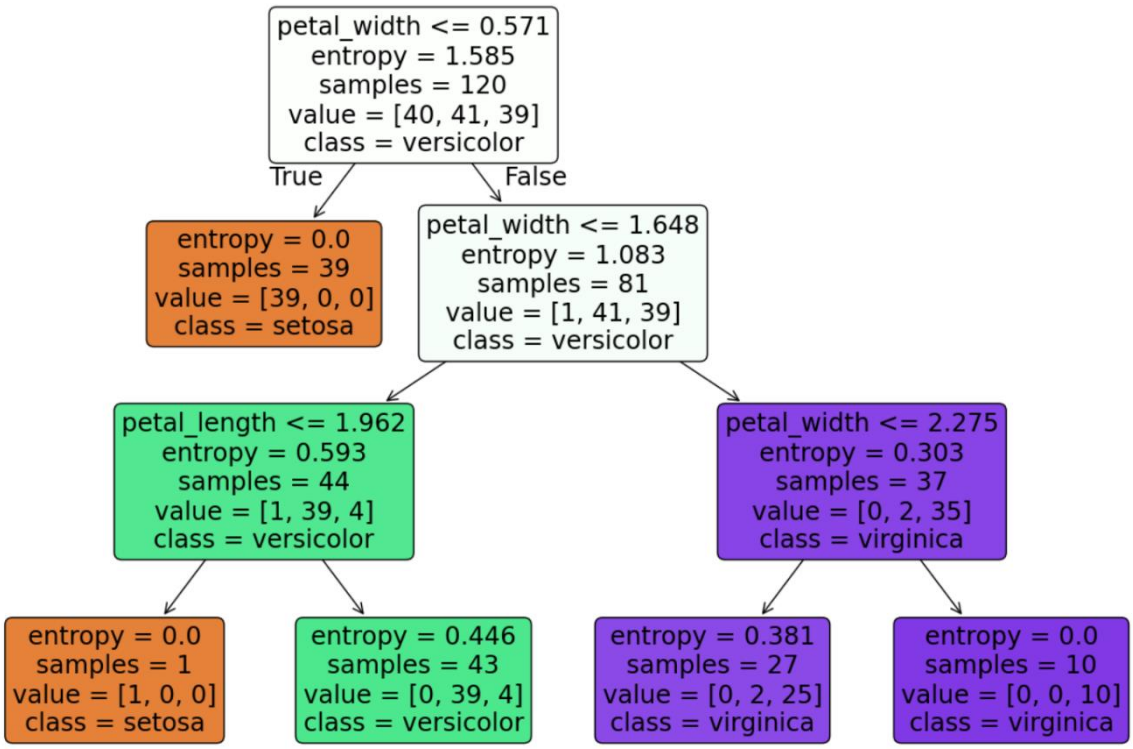
SOURCE CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)
print(X.head())
print("Target classes:", wine.target_names)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
plt.figure(figsize=(16, 10))
plot_tree(clf, filled=True, feature_names=wine.feature_names,
class_names=wine.target_names)
plt.title("Decision Tree for Wine Classification")
plt.show()
```

OUTPUT:

```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0
Target classes: ['class_0' 'class_1' 'class_2']


Accuracy: 0.9444444444444444

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.93      0.93        14
           1       0.93      1.00      0.97        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

Decision Tree Visualization



RESULT:

SOURCE CODE:

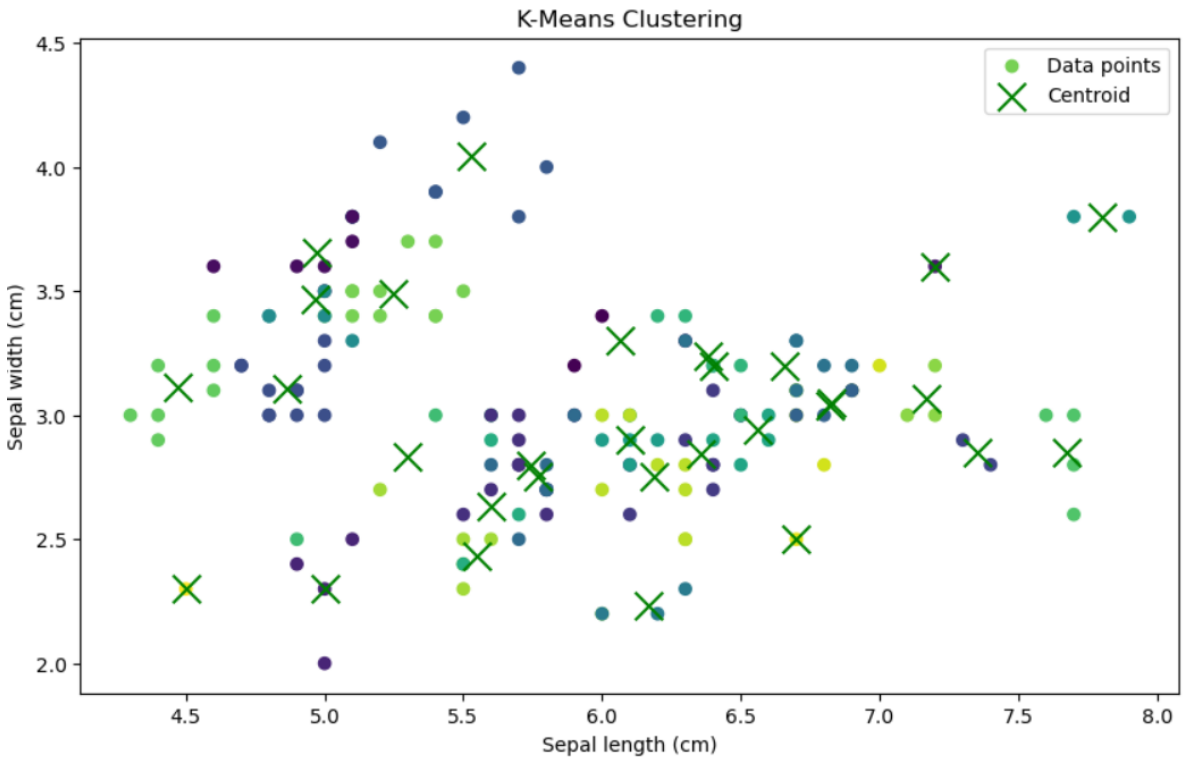
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

dataset=pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/refs/heads/master/iris.csv')

x=dataset.drop('species',axis=1).values

dataset

kmeans=KMeans(n_clusters=30,random_state=42)

kmeans.fit(x)

cl=kmeans.labels_

ce=kmeans.cluster_centers_

print("Type of X:",type(x),"Shape of X:",x.shape)

print("Type of cl:",type(cl),"Shape of cl:",cl.shape)

print("Type of ce:",type(ce),"Shape of ce:",ce.shape)

plt.figure(figsize=(10,6))

plt.scatter(x[:,0],x[:,1],c=cl,cmap='viridis',label='Data points')

plt.scatter(ce[:,0],ce[:,1],c='green',marker='x',s=200,label='Centroid')

plt.xlabel("Sepal length (cm)")

plt.ylabel("Sepal width (cm)")

plt.title("K-Means Clustering")

plt.legend()

plt.show()
```

OUTPUT:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
Type of X: <class 'numpy.ndarray'> Shape of X: (150, 4)
Type of cl: <class 'numpy.ndarray'> Shape of cl: (150,)
Type of ce: <class 'numpy.ndarray'> Shape of ce: (30, 4)
```


K-Means Clustering
Data points
Centroid
Sepal width (cm)
Sepal length (cm)
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

RESULT:

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape((60000, 28, 28, 1)) / 255.0
x_test = x_test.reshape((10000, 28, 28, 1)) / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
import matplotlib.pyplot as plt
history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.xlabel('Epoch')
```

In [2]:

```
plt.ylabel('Accuracy')
plt.legend()
plt.show()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

OUTPUT:

Epoch 1/5

844/844 [==========] - 30s 34ms/step - loss: 0.1974 - accuracy: 0.9405 - val_loss: 0.0651 - val_accuracy: 0.9817

Epoch 2/5

844/844 [==========] - 28s 33ms/step - loss: 0.0560 - accuracy: 0.9827 - val_loss: 0.0603 - val_accuracy: 0.9822

Epoch 3/5

844/844 [==========] - 29s 34ms/step - loss: 0.0387 - accuracy: 0.9879 - val_loss: 0.0453 - val_accuracy: 0.9875

Epoch 4/5

844/844 [==========] - 28s 33ms/step - loss: 0.0306 - accuracy: 0.9902 - val_loss: 0.0378 - val_accuracy: 0.9908

Epoch 5/5

844/844 [==========] - 28s 33ms/step - loss: 0.0251 - accuracy: 0.9920 - val_loss: 0.0334 - val_accuracy: 0.9905

313/313 [==========] - 2s 5ms/step - loss: 0.0306 - accuracy: 0.9897

Test accuracy: 0.9897000193595886

Epoch 1/5

844/844 [==========] - 28s 33ms/step - loss: 0.0207 - accuracy: 0.9930 - val_loss: 0.0337 - val_accuracy: 0.9920

Epoch 2/5

844/844 [==========] - 26s 31ms/step - loss: 0.0175 - accuracy: 0.9942 - val_loss: 0.0310 - val_accuracy: 0.9932

Epoch 3/5

844/844 [==========] - 26s 30ms/step - loss: 0.0139 - accuracy: 0.9954 - val_loss: 0.0407 - val_accuracy: 0.9915

Epoch 4/5

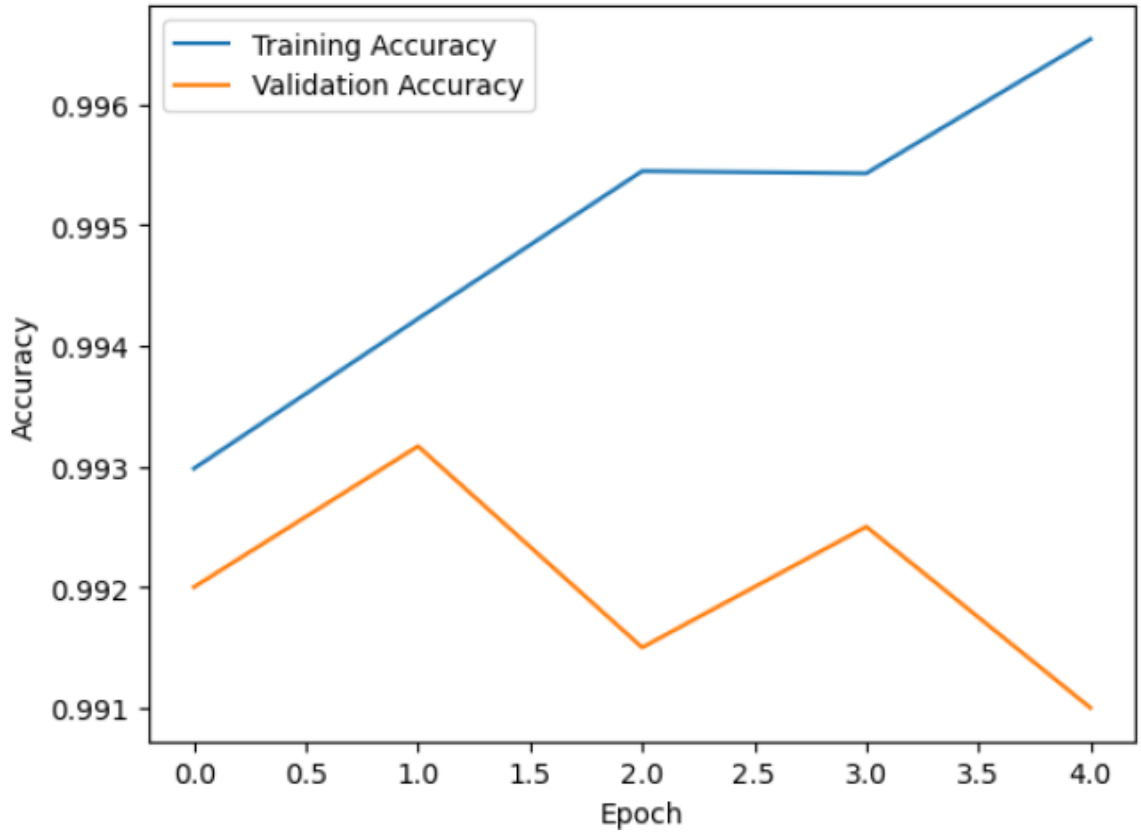
844/844 [==========] - 26s 30ms/step - loss: 0.0131 - accuracy: 0.9954 - val_loss: 0.0403 - val_accuracy: 0.9925

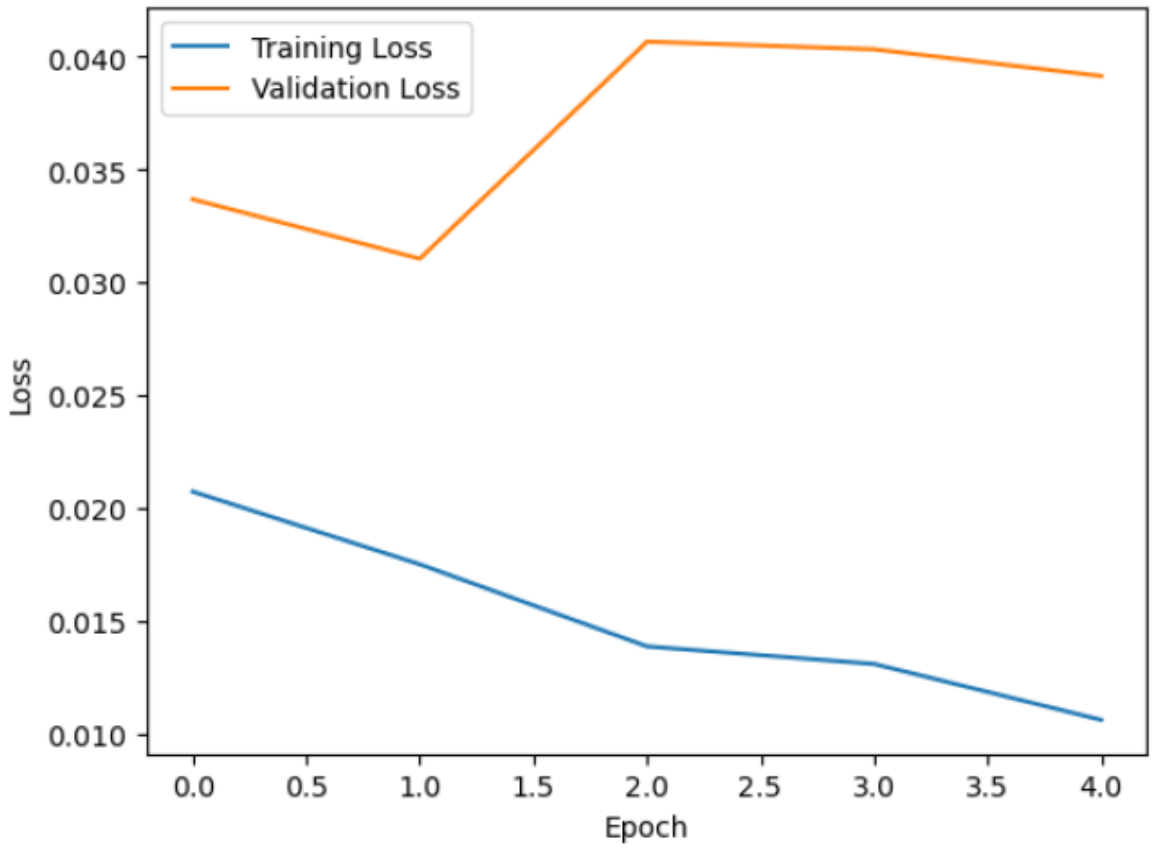
Epoch 5/5

844/844 [==========] - 26s 30ms/step - loss: 0.0106 - accuracy: 0.9965 - val_loss: 0.0391 - val_accuracy: 0.9910

313/313 [==========] - 2s 5ms/step - loss: 0.0379 - accuracy: 0.9902

Test accuracy: 0.9901999831199646


Training Accuracy
Validation Accuracy
Accuracy
0.996
0.995
0.994
0.993
0.992
0.991
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


Training Loss
Validation Loss
Loss
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch

RESULT:

SOURCE CODE:

```
import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.datasets import mnist

import matplotlib.pyplot as plt

(x_train, _), (x_test, _) = mnist.load_data()

x_train = x_train.reshape((60000, 784)) / 255.0

x_test = x_test.reshape((10000, 784)) / 255.0

encoding_dim = 32 # Dimension of the encoded representation

input_img = tf.keras.Input(shape=(784,))

encoded = layers.Dense(encoding_dim, activation='relu')(input_img)

decoded = layers.Dense(784, activation='sigmoid')(encoded)

autoencoder = models.Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

autoencoder.fit(x_train, x_train, epochs=10, batch_size=256, shuffle=True,
validation_data=(x_test, x_test))

encoded_imgs = autoencoder.predict(x_test)

decoded_imgs = autoencoder.predict(x_test)

n = 10

plt.figure(figsize=(20, 4))

for i in range(n):

        plt.subplot(2, n, i + 1)

        plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

        plt.axis('off')

        plt.subplot(2, n, i + 1 + n)

        plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')

        plt.axis('off')

plt.show()
```

OUTPUT:

Epoch 1/10

235/235 [==========] - 2s 6ms/step - loss: 0.2761 - val_loss: 0.1917

Epoch 2/10

235/235 [==========] - 1s 6ms/step - loss: 0.1736 - val_loss: 0.1562

Epoch 3/10

235/235 [==========] - 1s 6ms/step - loss: 0.1459 - val_loss: 0.1345

Epoch 4/10

235/235 [==========] - 1s 6ms/step - loss: 0.1291 - val_loss: 0.1218

Epoch 5/10

235/235 [==========] - 1s 6ms/step - loss: 0.1186 - val_loss: 0.1132

Epoch 6/10

235/235 [==========] - 2s 6ms/step - loss: 0.1112 - val_loss: 0.1069

Epoch 7/10

235/235 [==========] - 1s 6ms/step - loss: 0.1057 - val_loss: 0.1024

Epoch 8/10

235/235 [==========] - 1s 6ms/step - loss: 0.1018 - val_loss: 0.0991

Epoch 9/10

235/235 [==========] - 1s 6ms/step - loss: 0.0991 - val_loss: 0.0968

Epoch 10/10

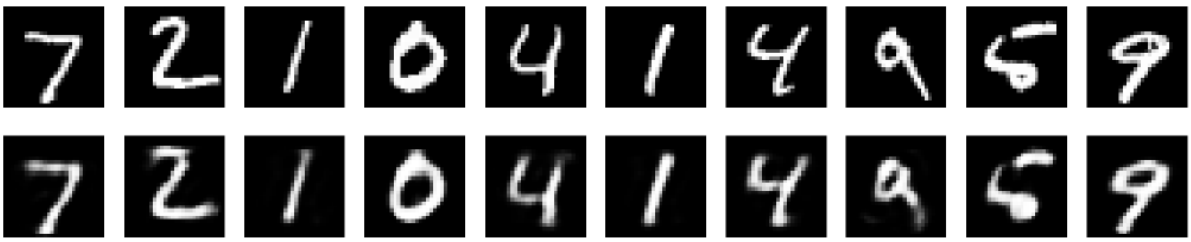
235/235 [==========] - 1s 6ms/step - loss: 0.0973 - val_loss: 0.0953

313/313 [==========] - 1s 1ms/step

313/313 [==========] - 0s 1ms/step

313/313 [==========] - 1s 1ms/step

313/313 [==========] - 0s 1ms/step



RESULT:

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
import matplotlib.pyplot as plt
max_features = 10000 # Number of words to consider as features
maxlen = 100 # Maximum length of sequences
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
embedding_dim = 128
model = models.Sequential()
model.add(layers.Embedding(max_features, embedding_dim, input_length=maxlen))
model.add(layers.LSTM(128))
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=5, batch_size=32, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
plt.figure(figsize=(8, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz

17464789/17464789 [==========] - 2s 0us/step

Epoch 1/5

704/704 [==========] - 75s 101ms/step - loss: 0.4209 - accuracy: 0.8079 - val_loss: 0.4025 - val_accuracy: 0.8332

Epoch 2/5

704/704 [==========] - 82s 117ms/step - loss: 0.2637 - accuracy: 0.8925 - val_loss: 0.4121 - val_accuracy: 0.8376

Epoch 3/5

704/704 [==========] - 85s 120ms/step - loss: 0.1767 - accuracy: 0.9318 - val_loss: 0.4401 - val_accuracy: 0.8368

Epoch 4/5

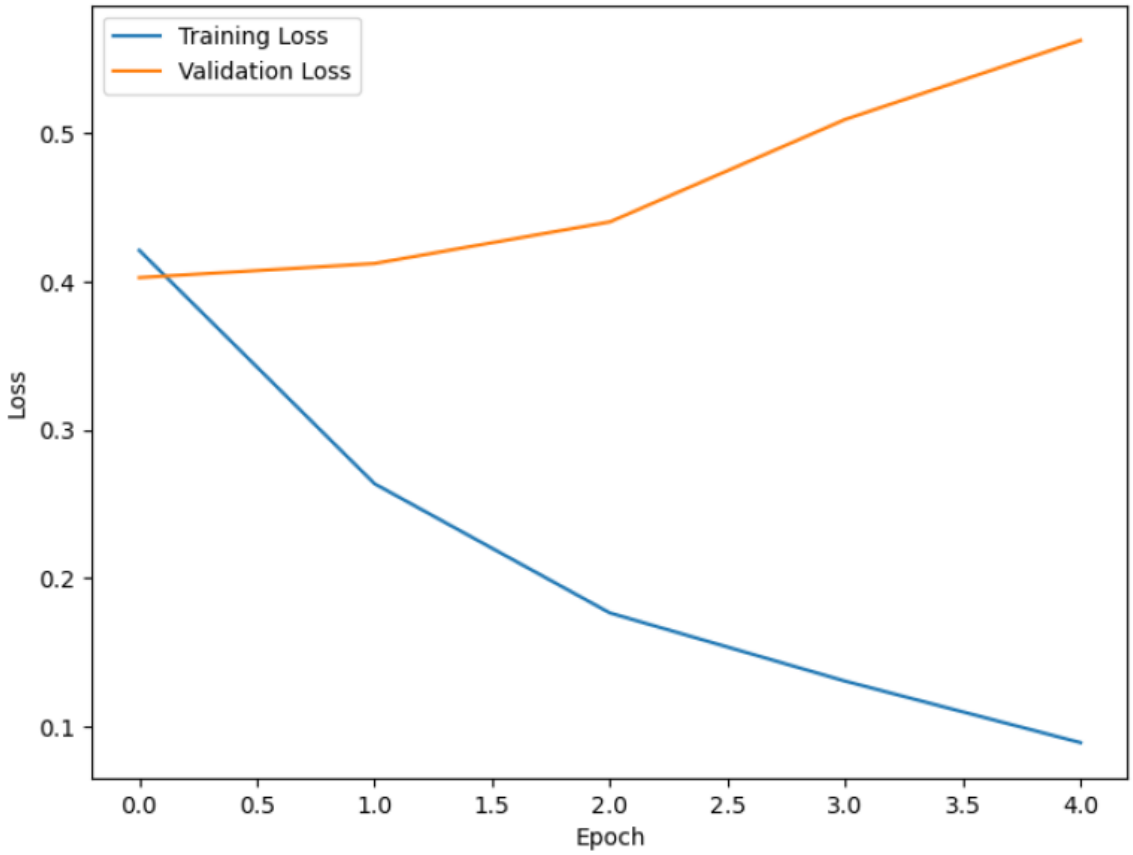
704/704 [==========] - 83s 119ms/step - loss: 0.1308 - accuracy: 0.9527 - val_loss: 0.5091 - val_accuracy: 0.8320

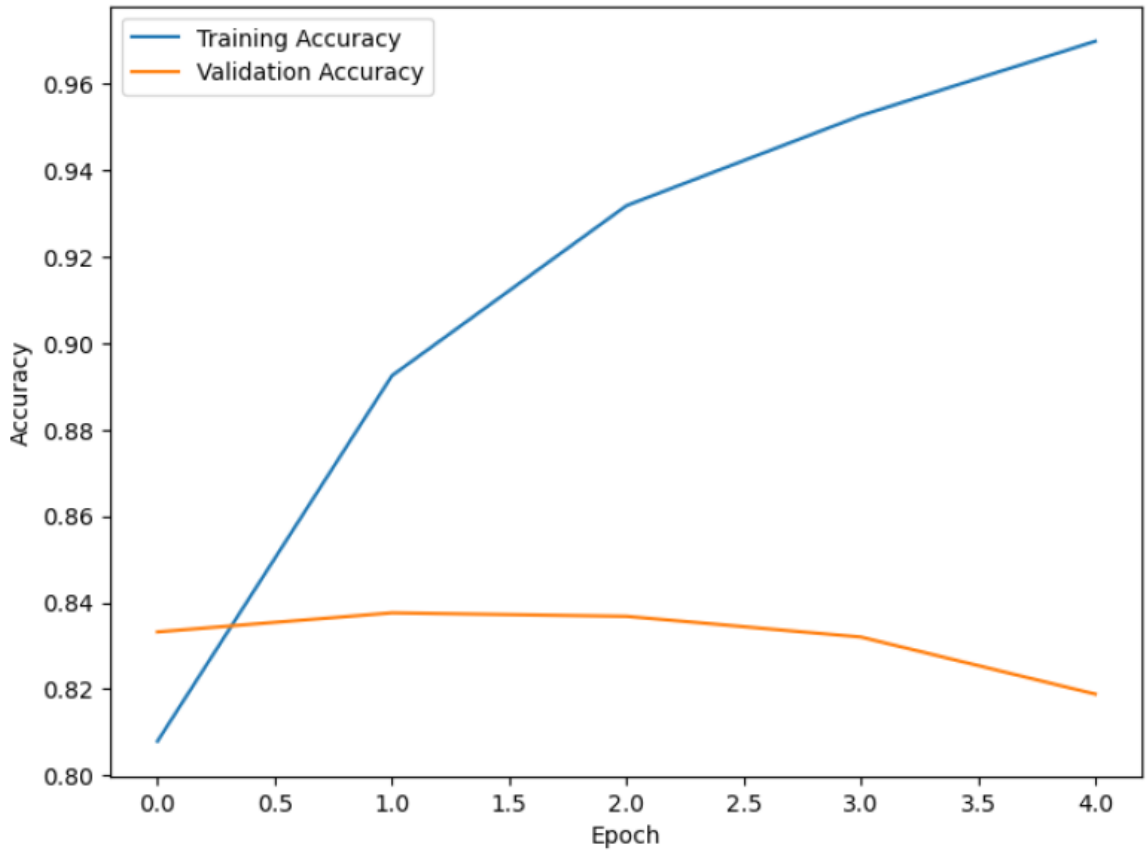
Epoch 5/5

704/704 [==========] - 81s 115ms/step - loss: 0.0894 - accuracy: 0.9699 - val_loss: 0.5623 - val_accuracy: 0.8188

782/782 [==========] - 30s 38ms/step - loss: 0.5520 - accuracy: 0.8187

Test accuracy: 0.8186799883842468


Training Loss
Validation Loss
Loss
0.5
0.4
0.3
0.2
0.1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch


Training Accuracy
Validation Accuracy
Accuracy
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0.80
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch

RESULT: